TESTES UNITÁRIOS

Até agora, você provavelmente tem testado seu próprio código usando instruções de impressão. Alternativamente, você pode ter contado com o CS50 para testar seu código para você! É mais comum na indústria escrever código para testar seus próprios programas. Na janela do console, digite código calculator.py. Note que você pode ter codificado este arquivo em uma aula anterior. No editor de texto, certifique-se de que seu código apareça da seguinte forma:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))


def square(n):
    return n * n


if __name__ == "__main__":
    main()
```

Observe que você poderia testar plausivelmente o código acima por conta própria usando alguns números óbvios, como 2. No entanto, considere porque você pode querer criar um teste que garanta que o código acima funcione adequadamente.

Seguindo a convenção, vamos criar um programa de teste digitando código test_calculator.py e modificando seu código no editor de texto da seguinte forma:

```
from calculator import square


def main():
    test_square()


def test_square():
```

```
    if square(2) != 4:
        print("2 squared was not 4")
    if square(3) != 9:
        print("3 squared was not 9")


if __name__ == "__main__":
    main()
```

Observe que estamos importando a função square de square.py na primeira linha de código. Por convenção, estamos criando uma função chamada test_square. Dentro dessa função, definimos algumas condições para testar.

Na janela do console, digite python test_calculator.py. Você perceberá que nada está sendo exibido. Pode ser que tudo esteja funcionando bem! Alternativamente, pode ser que nossa função de teste não tenha descoberto um dos "casos extremos" que poderiam produzir um erro.

Atualmente, nosso código testa duas condições. Se quiséssemos testar muitas mais condições, nosso código de teste poderia facilmente se tornar volumoso.

ASSERT

O comando assert do Python nos permite dizer ao compilador que algo, alguma assertiva, é verdadeira. Podemos aplicar isso ao nosso código de teste da seguinte maneira:

```
rom calculator import square


def main():
    test_square()


def test_square():
    assert square(2) == 4
    assert square(3) == 9
```

```
if __name__ == "__main__":
    main()
```

Observe que estamos afirmando definitivamente o que square(2) e square(3) deveriam ser iguais. Nosso código é reduzido de quatro linhas de teste para duas.

Podemos quebrar intencionalmente nosso código de calculadora modificando-o da seguinte maneira:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))


def square(n):
    return n + n


if __name__ == "__main__":
    main()
```

Observe que mudamos o operador * para um + na função square.

Agora, executando python test_square.py na janela do console, você perceberá que um AssertionError é gerado pelo compilador. Essencialmente, o compilador está nos informando que uma de nossas condições não foi atendida.

Um dos desafios que estamos enfrentando agora é que nosso código poderia se tornar ainda mais pesado se quiséssemos fornecer uma saída de erro mais descritiva para nossos usuários. Plausivelmente, poderíamos codificar da seguinte maneira:

```
from calculator import square


def main():
```

```
    test_square()


def test_square():
    try:
        assert square(2) == 4
    except AssertionError:
        print("2 squared is not 4")
    try:
        assert square(3) == 9
    except AssertionError:
        print("3 squared is not 9")
    try:
        assert square(-2) == 4
    except AssertionError:
        print("-2 squared is not 4")
    try:
        assert square(-3) == 9
    except AssertionError:
        print("-3 squared is not 9")
    try:
        assert square(0) == 0
    except AssertionError:
        print("0 squared is not 0")


if __name__ == "__main__":
    main()
```

Observe que executar este código produzirá vários erros. No entanto, não está produzindo todos os erros acima. Esta é uma boa ilustração de que vale a pena testar vários casos para que você possa pegar situações em que haja erros de codificação.

O código acima ilustra um grande desafio: como poderíamos facilitar o teste do seu código sem dezenas de linhas de código como as acima?

Você pode aprender mais na documentação do assert do Python.

PYTEST

Pytest é uma biblioteca de terceiros que permite testar unidades de seu programa. Ou seja, você pode testar suas funções dentro do seu programa. Para utilizar o pytest, digite pip install pytest na sua janela do console.

Antes de aplicar o pytest ao nosso próprio programa, modifique sua função test_calculator da seguinte forma:

```
from calculator import square


def test_assert():
    assert square(2) == 4
    assert square(3) == 9
    assert square(-2) == 4
    assert square(-3) == 9
    assert square(0) == 0
```

Observe como o código acima assertiva todas as condições que queremos testar.

O pytest nos permite executar nosso programa diretamente através dele, de modo que podemos visualizar mais facilmente os resultados de nossas condições de teste. Na janela do terminal, digite pytest test_calculator.py. Você perceberá imediatamente que a saída será fornecida. Observe o F vermelho perto do topo da saída, indicando que algo em seu código falhou. Além disso, observe que o E vermelho fornece algumas dicas sobre os erros em seu programa calculator.py. Com base na saída, você pode imaginar um cenário em que 3 * 3 gerou uma saída de 6 em vez de 9. Com base nos resultados deste teste, podemos corrigir nosso código calculator.py da seguinte forma:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))


def square(n):
    return n * n


if __name__ == "__main__":
    main()
```

Observe que mudamos o operador + para um * na função square, retornando-o a um estado funcional.

Ao executar novamente pytest test_calculator.py, observe como nenhum erro é produzido. Parabéns!

No momento, não é ideal que o pytest pare de ser executado após o primeiro teste falhado. Novamente, vamos retornar nosso código calculator.py ao seu estado quebrado:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))


def square(n):
    return n + n


if __name__ == "__main__":
    main()
```

Observe que mudamos o operador * para um + na função square, retornando-o a um estado quebrado.

Para melhorar nosso código de teste, vamos modificar test_calculator.py para dividir o código em diferentes grupos de testes:

```
from calculator import square


def test_positive():
    assert square(2) == 4
    assert square(3) == 9



def test_negative():
    assert square(-2) == 4
    assert square(-3) == 9



def test_zero():
    assert square(0) == 0
```

Observe que dividimos os mesmos cinco testes em três funções diferentes. Estruturas de teste como o pytest executarão cada função, mesmo se houver uma falha em uma delas. Ao executar novamente pytest test_calculator.py, você perceberá que muitos mais erros estão sendo exibidos. Mais saída de erro permite que você explore mais profundamente o que pode estar produzindo os problemas dentro do seu código.

Tendo melhorado nosso código de teste, retorne seu código calculator.py ao estado de funcionamento completo:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))



def square(n):
    return n * n
```

```
if __name__ == "__main__":
    main()
```

Observe que mudamos o operador + para um * na função square, retornando-o a um estado funcional.

Ao executar novamente pytest test_calculator.py, você perceberá que nenhum erro é encontrado.

Finalmente, podemos testar se nosso programa trata exceções. Vamos modificar test_calculator.py para fazer exatamente isso.

```
import pytest

from calculator import square

def test_positive():
    assert square(2) == 4
    assert square(3) == 9

def test_negative():
    assert square(-2) == 4
    assert square(-3) == 9

def test_zero():
    assert square(0) == 0

def test_str():
    with pytest.raises(TypeError):
        square("cat")
```

Observe que, em vez de usar assert, estamos aproveitando uma função dentro da própria biblioteca pytest chamada raises, que permite expressar que você espera que um erro seja gerado. Precisamos ir para o topo de nosso programa e adicionar import pytest e depois chamar pytest.raises com o tipo de erro que estamos esperando.

Novamente, ao executar pytest test_calculator.py, você perceberá que nenhum erro é encontrado.

Em resumo, cabe a você, como programador, definir quantas condições de teste achar adequado!

Você pode aprender mais na documentação do pytest do Pytest.

TESTANDO STRINGS

Voltando no tempo, considere o seguinte código hello.py:

```
def main():
    name = input("What's your name? ")
    hello(name)


def hello(to="world"):
    print("hello,", to)


if __name__ == "__main__":
    main()
```

Olhando para este código, você acha que esta abordagem de teste funcionará bem? Por que este teste pode não funcionar bem? Observe que a função hello em hello.py imprime algo: ou seja, ela não retorna um valor!

Podemos alterar nossa função hello dentro de hello.py da seguinte maneira:

```
from hello import hello


def test_hello():
```

```
assert hello("David") == "hello, David"
assert hello() == "hello, world"
```

Observe que mudamos nossa função hello para retornar uma string. Isso efetivamente significa que agora podemos usar o pytest para testar a função hello.

Ao executar pytest test_hello.py, nosso código passará em todos os testes!

Assim como nosso caso de teste anterior nesta lição, podemos separar nossos testes separadamente:

```
def main():
    name = input("What's your name? ")
    print(hello(name))


def hello(to="world"):
    return f"hello, {to}"


if __name__ == "__main__":
    main()
```

Observe que alteramos nossa função hello para retornar uma string. Isso significa efetivamente que agora podemos usar pytest para testar a função hello.

Executando pytest test_hello.py, nosso código passará em todos os testes!

Tal como acontece com o nosso caso de teste anterior nesta lição, podemos dividir nossos testes separadamente:

```
from hello import hello


def test_default():
    assert hello() == "hello, world"
```

```
def test_argument():
    assert hello("David") == "hello, David"
```

Observe que o código acima separa nosso teste em várias funções de modo que todas serão executadas, mesmo se um erro for produzido.

ORGANIZANDO TESTES EM PASTAS

Testar código usando vários testes de unidade é tão comum que você tem a capacidade de executar uma pasta inteira de testes com um único comando. Primeiro, na janela do terminal, execute mkdir test para criar uma pasta chamada test. Em seguida, para criar um teste dentro dessa pasta, digite na janela do terminal código test/test_hello.py. Observe que test/ instrui o terminal a criar test_hello.py na pasta chamada test. No editor de texto, modifique o arquivo para incluir o seguinte código:

```
from hello import hello

def test_default():
    assert hello() == "hello, world"


def test_argument():
    assert hello("David") == "hello, David"
```

Observe que estamos criando um teste como fizemos antes.

O pytest não nos permitirá executar testes como uma pasta apenas com este arquivo (ou um conjunto inteiro de arquivos) sozinho sem um arquivo **init** especial. Na janela do seu terminal, crie este arquivo digitando código test/**init**.py. Observe o test/ como antes, bem como os sublinhados duplos em cada lado de init. Mesmo deixando este arquivo **init**.py vazio, o pytest é informado de que a pasta inteira contendo **init**.py tem testes que podem ser executados. Agora, digitando pytest test no terminal, você pode executar toda a pasta de teste de código.

Você pode aprender mais na documentação do pytest sobre os mecanismos de importação.

RESUMINDO

Testar seu código é uma parte natural do processo de programação. Testes de unidade permitem que você teste aspectos específicos do seu código. Você pode criar seus próprios programas que testam seu código. Alternativamente, você pode utilizar frameworks como o pytest para executar seus testes de unidade para você.